

Lecture Notes: Backpropagation in a One-Hidden-Layer Neural Network

Based on the Core Ideas of Chapter 2 of *Neural Networks and Deep Learning*

Dr. Adnan Arefeen

Example Network Setup

We use a simple example with two input features:

$$x_1 = 1, \quad x_2 = 0.$$

The desired target output is:

$$y = 1.$$

We use a neural network with:

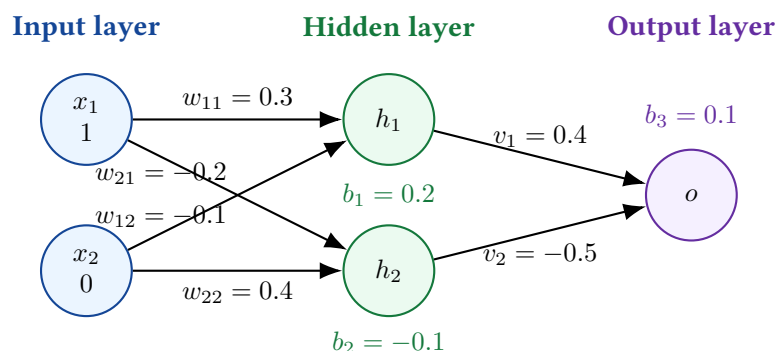
- two input neurons: x_1, x_2 ,
- two hidden neurons: h_1, h_2 ,
- one output neuron: o .

So the network structure is:

$$2 \longrightarrow 2 \longrightarrow 1$$

That means two inputs, one hidden layer with two neurons, and one output neuron.

Network Diagram



Activation Function: Sigmoid

We use the sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

The sigmoid squashes any real number into the range $(0, 1)$.

Initial Parameters

Parameter Type	Parameter	Value
Input-to-hidden weight	w_{11}	0.3
Input-to-hidden weight	w_{12}	-0.1
Input-to-hidden weight	w_{21}	-0.2
Input-to-hidden weight	w_{22}	0.4
Hidden-layer bias	b_1	0.2
Hidden-layer bias	b_2	-0.1
Hidden-to-output weight	v_1	0.4
Hidden-to-output weight	v_2	-0.5
Output bias	b_3	0.1
Learning rate	η	0.5

Table 1: Initial parameters of the one-hidden-layer neural network.

Important formula

The derivative of the sigmoid function is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

This formula is very useful because, during backpropagation, once we know the activation value $a = \sigma(z)$, we can compute the derivative as:

$$\sigma'(z) = a(1 - a).$$

Derivation of the Sigmoid Derivative

Start with:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Rewrite it as:

$$\sigma(z) = (1 + e^{-z})^{-1}.$$

Differentiate using the chain rule:

$$\frac{d}{dz}\sigma(z) = \frac{d}{dz}(1 + e^{-z})^{-1}.$$

Let:

$$u = 1 + e^{-z}.$$

Then:

$$\sigma(z) = u^{-1}.$$

Using:

$$\frac{d}{dz}u^{-1} = -u^{-2}\frac{du}{dz},$$

and:

$$\frac{du}{dz} = \frac{d}{dz}(1 + e^{-z}) = -e^{-z},$$

we get:

$$\sigma'(z) = -(1 + e^{-z})^{-2}(-e^{-z}).$$

So:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Now show that this equals $\sigma(z)(1 - \sigma(z))$.

We know:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Now:

$$1 - \sigma(z) = 1 - \frac{1}{1 + e^{-z}}.$$

Use the same denominator:

$$1 - \sigma(z) = \frac{1 + e^{-z}}{1 + e^{-z}} - \frac{1}{1 + e^{-z}}.$$

Therefore:

$$1 - \sigma(z) = \frac{e^{-z}}{1 + e^{-z}}.$$

Now multiply:

$$\sigma(z)(1 - \sigma(z)) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}.$$

Thus:

$$\sigma(z)(1 - \sigma(z)) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

But we already found:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Therefore:

$$\boxed{\sigma'(z) = \sigma(z)(1 - \sigma(z))}.$$

Forward Pass

The forward pass computes the prediction $\hat{y}$.

Hidden Neuron 1

The weighted input to hidden neuron 1 is:

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1.$$

Substitute the values:

$$z_1 = (0.3)(1) + (-0.1)(0) + 0.2.$$

$$z_1 = 0.3 + 0 + 0.2 = 0.5.$$

Now apply sigmoid:

$$a_1 = \sigma(z_1) = \sigma(0.5).$$

$$a_1 = 0.6225.$$

So:

$$\boxed{a_1 = 0.6225}.$$

Hidden Neuron 2

The weighted input to hidden neuron 2 is:

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2.$$

Substitute the values:

$$z_2 = (-0.2)(1) + (0.4)(0) + (-0.1).$$

$$z_2 = -0.2 + 0 - 0.1 = -0.3.$$

Now apply sigmoid:

$$a_2 = \sigma(z_2) = \sigma(-0.3).$$

$$a_2 = 0.4256.$$

So:

$$\boxed{a_2 = 0.4256}.$$

Output Neuron

The weighted input to the output neuron is:

$$z_3 = v_1 a_1 + v_2 a_2 + b_3.$$

Substitute the values:

$$z_3 = (0.4)(0.6225) + (-0.5)(0.4256) + 0.1.$$

$$z_3 = 0.2490 - 0.2128 + 0.1.$$

$$z_3 = 0.1362.$$

Apply sigmoid:

$$a_3 = \sigma(z_3) = \sigma(0.1362).$$

$$a_3 = 0.5340.$$

Therefore, the prediction is:

$$\hat{y} = a_3 = 0.5340.$$

Interpretation

The target is $y = 1$, but the network predicts $\hat{y} = 0.5340$. The prediction is too low, so the network must adjust its parameters to increase the output.

Cost Function

We use the quadratic cost function:

$$C = \frac{1}{2}(\hat{y} - y)^2.$$

Since $\hat{y} = 0.5340$ and $y = 1$:

$$C = \frac{1}{2}(0.5340 - 1)^2.$$

$$C = \frac{1}{2}(-0.4660)^2.$$

$$C = 0.1086.$$

So:

$$C = 0.1086.$$

Backpropagation Overview

Backpropagation computes how much each parameter contributed to the error.

For each neuron, we compute an error term δ .

Core backpropagation idea

For a weight connected into a neuron:

$$\frac{\partial C}{\partial w} = \text{error of receiving neuron} \times \text{input to that weight}.$$

For example:

$$\frac{\partial C}{\partial v_1} = \delta_3 a_1.$$

Here:

- δ_3 is the error of the output neuron,
- a_1 is the input coming into weight v_1 .

Proof:

$$\begin{aligned}\frac{\partial C}{\partial v_1} &= \frac{\partial C}{\partial z_3} \cdot \frac{\partial z_3}{\partial v_1} \\ &= \delta_3 \cdot \frac{\partial}{\partial v_1} (v_1 a_1 + v_2 a_2 + b_3) \\ &= \delta_3 \cdot a_1 \\ \therefore \frac{\partial C}{\partial v_1} &= \delta_3 a_1\end{aligned}$$

Output Layer Error

$$\begin{aligned}\delta_3 &= \frac{\partial C}{\partial z_3} \\ &= \frac{\partial C}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \\ &= \frac{\partial}{\partial a_3} \left[\frac{1}{2} (a_3 - y)^2 \right] \cdot \frac{\partial}{\partial z_3} [\sigma(z_3)] \\ &= (a_3 - y) \cdot \sigma'(z_3) \\ \therefore \delta_3 &= (a_3 - y) \sigma'(z_3) \\ \delta_3 &= (a_3 - y) a_3 (1 - a_3)\end{aligned}$$

The output error is:

Important formula

$$\delta_3 = (a_3 - y) \sigma'(z_3) \quad \delta_3 = (a_3 - y) a_3 (1 - a_3)$$

First compute:

$$a_3 - y = 0.5340 - 1 = -0.4660.$$

Now compute the sigmoid derivative:

$$\sigma'(z_3) = a_3(1 - a_3).$$

$$\sigma'(z_3) = 0.5340(1 - 0.5340).$$

$$\sigma'(z_3) = 0.5340(0.4660).$$

$$\sigma'(z_3) = 0.2488.$$

Therefore:

$$\delta_3 = (-0.4660)(0.2488).$$

$$\boxed{\delta_3 = -0.1160}.$$

Interpretation of negative output error

A negative δ_3 means the output should increase. Since the current prediction 0.5340 is below the target 1, this makes sense.

Hidden Layer Errors

Now we send the output error backward to the hidden layer.

Hidden Neuron 1 Error

Hidden neuron h_1 affects the cost C through the following path:

$$\begin{aligned} z_1 &\rightarrow a_1 \rightarrow z_3 \rightarrow a_3 \rightarrow C \\ \delta_1 &= \frac{\partial C}{\partial z_1} \\ &= \frac{\partial C}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \\ &= \delta_3 \cdot \frac{\partial}{\partial a_1} (v_1 a_1 + v_2 a_2 + b_3) \cdot \frac{\partial}{\partial z_1} (\sigma(z_1)) \\ &= \delta_3 \cdot v_1 \cdot \sigma'(z_1) \\ \therefore \delta_1 &= v_1 \delta_3 \sigma'(z_1) \end{aligned}$$

The error term for hidden neuron 1 is:

$$\delta_1 = v_1 \delta_3 \sigma'(z_1).$$

We know:

$$v_1 = 0.4, \quad \delta_3 = -0.1160.$$

Now compute:

$$\sigma'(z_1) = a_1(1 - a_1).$$

$$\sigma'(z_1) = 0.6225(1 - 0.6225).$$

$$\sigma'(z_1) = 0.6225(0.3775).$$

$$\sigma'(z_1) = 0.2350.$$

So:

$$\delta_1 = (0.4)(-0.1160)(0.2350).$$

$$\boxed{\delta_1 = -0.0109}.$$

Hidden Neuron 2 Error

Hidden neuron h_2 affects the cost C through the following path:

$$z_2 \rightarrow a_2 \rightarrow z_3 \rightarrow a_3 \rightarrow C$$

Therefore, the error term for hidden neuron h_2 is obtained using the chain rule.

$$\begin{aligned} \delta_2 &= \frac{\partial C}{\partial z_2} \\ &= \frac{\partial C}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \\ &= \delta_3 \cdot \frac{\partial}{\partial a_2} (v_1 a_1 + v_2 a_2 + b_3) \cdot \frac{\partial}{\partial z_2} (\sigma(z_2)) \\ &= \delta_3 \cdot v_2 \cdot \sigma'(z_2) \\ \therefore \delta_2 &= v_2 \delta_3 \sigma'(z_2) \end{aligned}$$

The error term for hidden neuron 2 is:

$$\delta_2 = v_2 \delta_3 \sigma'(z_2).$$

We know:

$$v_2 = -0.5, \quad \delta_3 = -0.1160.$$

Now compute:

$$\sigma'(z_2) = a_2(1 - a_2).$$

$$\sigma'(z_2) = 0.4256(1 - 0.4256).$$

$$\sigma'(z_2) = 0.4256(0.5744).$$

$$\sigma'(z_2) = 0.2445.$$

So:

$$\delta_2 = (-0.5)(-0.1160)(0.2445).$$

$$\boxed{\delta_2 = 0.0142}.$$

Gradients for Hidden-to-Output Weights

For the output-layer weights:

$$\frac{\partial C}{\partial v_j} = \delta_3 a_j.$$

Gradient for v_1

$$\frac{\partial C}{\partial v_1} = \delta_3 a_1.$$

$$\frac{\partial C}{\partial v_1} = (-0.1160)(0.6225).$$

$$\boxed{\frac{\partial C}{\partial v_1} = -0.0722}.$$

Gradient for v_2

$$\frac{\partial C}{\partial v_2} = \delta_3 a_2.$$

$$\frac{\partial C}{\partial v_2} = (-0.1160)(0.4256).$$

$$\boxed{\frac{\partial C}{\partial v_2} = -0.0493}.$$

Gradient for b_3

For a bias, the input is always 1, so:

$$\frac{\partial C}{\partial b_3} = \delta_3.$$

$$\boxed{\frac{\partial C}{\partial b_3} = -0.1160}.$$

Gradients for Input-to-Hidden Weights

For the input-to-hidden weights:

$$\frac{\partial C}{\partial w_{ij}} = \delta_i x_j.$$

Gradients for Hidden Neuron 1

For w_{11} :

$$\frac{\partial C}{\partial w_{11}} = \delta_1 x_1.$$

$$\frac{\partial C}{\partial w_{11}} = (-0.0109)(1).$$

$$\boxed{\frac{\partial C}{\partial w_{11}} = -0.0109}.$$

For w_{12} :

$$\frac{\partial C}{\partial w_{12}} = \delta_1 x_2.$$

$$\frac{\partial C}{\partial w_{12}} = (-0.0109)(0).$$

$$\boxed{\frac{\partial C}{\partial w_{12}} = 0}.$$

For b_1 :

$$\frac{\partial C}{\partial b_1} = \delta_1.$$

$$\boxed{\frac{\partial C}{\partial b_1} = -0.0109}.$$

Gradients for Hidden Neuron 2

For w_{21} :

$$\frac{\partial C}{\partial w_{21}} = \delta_2 x_1.$$

$$\frac{\partial C}{\partial w_{21}} = (0.0142)(1).$$

$$\boxed{\frac{\partial C}{\partial w_{21}} = 0.0142}.$$

For w_{22} :

$$\frac{\partial C}{\partial w_{22}} = \delta_2 x_2.$$

$$\frac{\partial C}{\partial w_{22}} = (0.0142)(0).$$

$$\boxed{\frac{\partial C}{\partial w_{22}} = 0}.$$

For b_2 :

$$\frac{\partial C}{\partial b_2} = \delta_2.$$

$$\frac{\partial C}{\partial b_2} = 0.0142.$$

Why are some gradients zero?

The gradients for w_{12} and w_{22} are zero because $x_2 = 0$. A weight only updates if its input is active. Since the input x_2 is zero in this training example, those two weights receive no update.

Gradient Descent Update Rule

The gradient descent update rule is:

$$\theta_{new} = \theta_{old} - \eta \frac{\partial C}{\partial \theta}.$$

Here:

- θ means any parameter: weight or bias,
- η is the learning rate,
- $\frac{\partial C}{\partial \theta}$ is the gradient.

In this example:

$$\eta = 0.5.$$

Updating the Hidden-to-Output Parameters

Update v_1

$$v_1^{new} = v_1^{old} - \eta \frac{\partial C}{\partial v_1}.$$

$$v_1^{new} = 0.4 - 0.5(-0.0722).$$

$$v_1^{new} = 0.4 + 0.0361.$$

$$v_1^{new} = 0.4361.$$

Update v_2

$$v_2^{new} = v_2^{old} - \eta \frac{\partial C}{\partial v_2}.$$

$$v_2^{new} = -0.5 - 0.5(-0.0493).$$

$$v_2^{new} = -0.5 + 0.0247.$$

$$v_2^{new} = -0.4753.$$

Update b_3

$$b_3^{new} = b_3^{old} - \eta \frac{\partial C}{\partial b_3}.$$

$$b_3^{new} = 0.1 - 0.5(-0.1160).$$

$$b_3^{new} = 0.1 + 0.0580.$$

$$\boxed{b_3^{new} = 0.1580}.$$

Updating the Input-to-Hidden Parameters

Update w_{11}

$$w_{11}^{new} = 0.3 - 0.5(-0.0109).$$

$$w_{11}^{new} = 0.3 + 0.00545.$$

$$\boxed{w_{11}^{new} = 0.3055}.$$

Update w_{12}

$$w_{12}^{new} = -0.1 - 0.5(0).$$

$$\boxed{w_{12}^{new} = -0.1000}.$$

Update b_1

$$b_1^{new} = 0.2 - 0.5(-0.0109).$$

$$b_1^{new} = 0.2 + 0.00545.$$

$$\boxed{b_1^{new} = 0.2055}.$$

Update w_{21}

$$w_{21}^{new} = -0.2 - 0.5(0.0142).$$

$$w_{21}^{new} = -0.2 - 0.0071.$$

$$\boxed{w_{21}^{new} = -0.2071}.$$

Update w_{22}

$$w_{22}^{new} = 0.4 - 0.5(0).$$

$$\boxed{w_{22}^{new} = 0.4000}.$$

Update b_2

$$b_2^{new} = -0.1 - 0.5(0.0142).$$

$$b_2^{new} = -0.1 - 0.0071.$$

$$\boxed{b_2^{new} = -0.1071}.$$

Summary Table of Gradients and Updates

Parameter	Old value	Gradient	New value
w_{11}	0.3000	-0.0109	0.3055
w_{12}	-0.1000	0.0000	-0.1000
b_1	0.2000	-0.0109	0.2055
w_{21}	-0.2000	0.0142	-0.2071
w_{22}	0.4000	0.0000	0.4000
b_2	-0.1000	0.0142	-0.1071
v_1	0.4000	-0.0722	0.4361
v_2	-0.5000	-0.0493	-0.4753
b_3	0.1000	-0.1160	0.1580

Complete Algorithm for One Training Example

Step 1: Input $x_1 = 1$, $x_2 = 0$, target $y = 1$.

Step 2: Compute hidden weighted sums z_1 and z_2 .

Step 3: Apply sigmoid to get hidden activations a_1 and a_2 .

Step 4: Compute output weighted sum z_3 .

Step 5: Apply sigmoid to get output prediction $a_3 = \hat{y}$.

Step 6: Compute cost $C = \frac{1}{2}(\hat{y} - y)^2$.

Step 7: Compute output error $\delta_3 = (a_3 - y)\sigma'(z_3)$.

Step 8: Compute hidden errors:

$$\delta_1 = v_1\delta_3\sigma'(z_1), \quad \delta_2 = v_2\delta_3\sigma'(z_2).$$

Step 9: Compute all gradients.

Step 10: Update all weights and biases using gradient descent.

Key Takeaways

Main lesson

Backpropagation is the process of calculating gradients efficiently using the chain rule. It starts from the output error and moves backward through the network to determine how each weight and bias should change.

- The forward pass gives the prediction.
- The cost function measures how wrong the prediction is.
- The output error tells us how the final neuron contributed to the mistake.
- The hidden errors tell us how hidden neurons contributed to the output error.
- Gradients tell us how to change each parameter.
- Gradient descent updates the parameters in the direction that reduces cost.

Compact Formula Sheet

*

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

*

Cost

$$C = \frac{1}{2}(\hat{y} - y)^2$$

*

Output Error

$$\delta_3 = (a_3 - y)\sigma'(z_3)$$

*

Hidden Errors

$$\delta_1 = v_1\delta_3\sigma'(z_1)$$

$$\delta_2 = v_2\delta_3\sigma'(z_2)$$

*

Gradients

$$\frac{\partial C}{\partial v_1} = \delta_3 a_1, \quad \frac{\partial C}{\partial v_2} = \delta_3 a_2, \quad \frac{\partial C}{\partial b_3} = \delta_3$$

The output neuron input is:

$$z_3 = v_1 a_1 + v_2 a_2 + b_3.$$

The output error term is defined as:

$$\delta_3 = \frac{\partial C}{\partial z_3}.$$

Now we compute the gradients for the hidden-to-output parameters.

$$\begin{aligned} \frac{\partial C}{\partial v_1} &= \frac{\partial C}{\partial z_3} \cdot \frac{\partial z_3}{\partial v_1} \\ &= \delta_3 \cdot \frac{\partial}{\partial v_1} (v_1 a_1 + v_2 a_2 + b_3) \\ &= \delta_3 \cdot a_1 \\ \therefore \frac{\partial C}{\partial v_1} &= \delta_3 a_1. \end{aligned}$$

Similarly, for v_2 :

$$\begin{aligned} \frac{\partial C}{\partial v_2} &= \frac{\partial C}{\partial z_3} \cdot \frac{\partial z_3}{\partial v_2} \\ &= \delta_3 \cdot \frac{\partial}{\partial v_2} (v_1 a_1 + v_2 a_2 + b_3) \\ &= \delta_3 \cdot a_2 \\ \therefore \frac{\partial C}{\partial v_2} &= \delta_3 a_2. \end{aligned}$$

For the output bias b_3 :

$$\begin{aligned} \frac{\partial C}{\partial b_3} &= \frac{\partial C}{\partial z_3} \cdot \frac{\partial z_3}{\partial b_3} \\ &= \delta_3 \cdot \frac{\partial}{\partial b_3} (v_1 a_1 + v_2 a_2 + b_3) \\ &= \delta_3 \cdot 1 \\ \therefore \frac{\partial C}{\partial b_3} &= \delta_3. \end{aligned}$$

Therefore, the output-layer gradients are:

$$\boxed{\frac{\partial C}{\partial v_1} = \delta_3 a_1, \quad \frac{\partial C}{\partial v_2} = \delta_3 a_2, \quad \frac{\partial C}{\partial b_3} = \delta_3}$$

$$\begin{aligned}\frac{\partial C}{\partial w_{11}} &= \delta_1 x_1, & \frac{\partial C}{\partial w_{12}} &= \delta_1 x_2, & \frac{\partial C}{\partial b_1} &= \delta_1 \\ \frac{\partial C}{\partial w_{21}} &= \delta_2 x_1, & \frac{\partial C}{\partial w_{22}} &= \delta_2 x_2, & \frac{\partial C}{\partial b_2} &= \delta_2\end{aligned}$$

The first hidden neuron input is:

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1.$$

The error term for hidden neuron h_1 is defined as:

$$\delta_1 = \frac{\partial C}{\partial z_1}.$$

Now we compute the gradients for the parameters connected to hidden neuron h_1 .

$$\begin{aligned}\frac{\partial C}{\partial w_{11}} &= \frac{\partial C}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}} \\ &= \delta_1 \cdot \frac{\partial}{\partial w_{11}} (w_{11}x_1 + w_{12}x_2 + b_1) \\ &= \delta_1 x_1.\end{aligned}$$

Similarly, for w_{12} :

$$\begin{aligned}\frac{\partial C}{\partial w_{12}} &= \frac{\partial C}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{12}} \\ &= \delta_1 \cdot \frac{\partial}{\partial w_{12}} (w_{11}x_1 + w_{12}x_2 + b_1) \\ &= \delta_1 x_2.\end{aligned}$$

For the bias b_1 :

$$\begin{aligned}\frac{\partial C}{\partial b_1} &= \frac{\partial C}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1} \\ &= \delta_1 \cdot \frac{\partial}{\partial b_1} (w_{11}x_1 + w_{12}x_2 + b_1) \\ &= \delta_1 \cdot 1 \\ &= \delta_1.\end{aligned}$$

Therefore, for hidden neuron h_1 :

$$\boxed{\frac{\partial C}{\partial w_{11}} = \delta_1 x_1, \quad \frac{\partial C}{\partial w_{12}} = \delta_1 x_2, \quad \frac{\partial C}{\partial b_1} = \delta_1}$$

Now consider the second hidden neuron input:

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2.$$

The error term for hidden neuron h_2 is defined as:

$$\delta_2 = \frac{\partial C}{\partial z_2}.$$

Now we compute the gradients for the parameters connected to hidden neuron h_2 .

$$\begin{aligned}\frac{\partial C}{\partial w_{21}} &= \frac{\partial C}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_{21}} \\ &= \delta_2 \cdot \frac{\partial}{\partial w_{21}} (w_{21}x_1 + w_{22}x_2 + b_2) \\ &= \delta_2 x_1.\end{aligned}$$

Similarly, for w_{22} :

$$\begin{aligned}\frac{\partial C}{\partial w_{22}} &= \frac{\partial C}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_{22}} \\ &= \delta_2 \cdot \frac{\partial}{\partial w_{22}} (w_{21}x_1 + w_{22}x_2 + b_2) \\ &= \delta_2 x_2.\end{aligned}$$

For the bias b_2 :

$$\begin{aligned}\frac{\partial C}{\partial b_2} &= \frac{\partial C}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2} \\ &= \delta_2 \cdot \frac{\partial}{\partial b_2} (w_{21}x_1 + w_{22}x_2 + b_2) \\ &= \delta_2 \cdot 1 \\ &= \delta_2.\end{aligned}$$

Therefore, for hidden neuron h_2 :

$\frac{\partial C}{\partial w_{21}} = \delta_2 x_1, \quad \frac{\partial C}{\partial w_{22}} = \delta_2 x_2, \quad \frac{\partial C}{\partial b_2} = \delta_2$
--

*

Update Rule

$$\theta_{new} = \theta_{old} - \eta \frac{\partial C}{\partial \theta}$$